Cited but Not Correct: Auditing Decisions, Citations and Claims in Evidence-Grounded LLM Advice

Anonymous ACL submission

Abstract

Language models increasingly recommend actions and justify them by citing evidence in the prompt. Such a rationale makes three promises: the action is good, its claims about the evidence are true, and the decision rests on what is cited. Evaluations rarely test all three or compare with what the evidence alone recommends. Our audit protocol gives an equally informed non-LLM selector the same frozen evidence, ablates and deletes evidence, verifies every quantitative claim of an explanation against it, and scores decisions on independently labeled held-out outcomes. We apply it to configuration advice on Linux kernel interventions with GroundKern, a testbed separating evidence from enforcement, and ten LLMs from 7B to frontier scale, with database settings as a second domain. The promises come apart. Evidence is necessary: without it, nine models mostly choose worse. It is also sufficient: with it, nine reproduce a table-lookup selector, optimal in the kernel task. Citations are valid but one-sided: 252 of 260 cited runs belong to the chosen configuration. Explanations are cited but not correct: 41 of 97 audited claims misstate the evidence, mostly alongside a correct decision; in the database domain, they misreport contrary evidence. Citation-specific reliance under deletion is rare and unstable.

1 Introduction

Language models are increasingly asked not only to answer questions but to recommend actions, and to justify each recommendation with citations to evidence supplied in the prompt. A rationale of this kind makes three distinct promises: that the recommended action is good, that what the rationale says about the evidence is true, and that the cited evidence is what the recommendation rests on. Evaluations usually check at most one of them. Work on attribution asks whether generated statements are supported by their sources (Rashkin et al., 2023; Gao et al., 2023b; Min et al., 2023; Liu et al., 2023), typically for question answering or summarization, where no downstream action is scored. Work on LLM decision-making scores the action (Yang et al., 2024; Lao et al., 2024; Giannakouris and Trummer, 2025) but treats the rationale as commentary. Neither asks what the evidence alone would have recommended.

Choice: `s50000_aall` (the best configuration) **Cited:** `training-table`, five retrieval runs (all valid IDs)

"... For s50000_aall, the P95 latencies across 10 runs are consistently low: 126.36, 140.554, 94.81, ... 87.347 µs. The maximum among these is 183.217 µs, which is lower than the lowest P95 observed for any other config ..."

Check: every quoted value is correct; the underlined comparison is false, since `s50000_aone` has a run at 85.033 µs.

Figure 1: A real full-context response from DeepSeek-V3.2 (seed 4089), abridged. The decision is correct and every citation is valid, yet the explanation misstates the evidence it cites. Across ten LLMs, 41 of 97 audited claims are misstated (§5).

We study these promises in configuration advice, an application where LLMs read manuals and measurements and propose database or kernel settings with natural-language justifications (Trummer, 2022; Lao et al., 2024; Giannakouris and Trummer, 2025; Chen et al., 2024; Lin et al., 2025). The setting is unusually well suited to auditing grounded rationales. The evidence is small, structured and numeric, so most claims about it can be checked exactly rather than by judging entailment. The quality of a decision can be measured on held-out runs, independently of the model and of any mechanism that enforces its advice. And the evidence can be handed unchanged to a conventional selector, which says what the evidence determines without a language model.

We ask three questions. *Added value*: does the model choose better than a conventional selector

that receives the same evidence, or does it only repackage that evidence? *Explanation correctness*: when an explanation cites valid evidence, does it state that evidence correctly? *Reliance*: does the cited evidence actually influence the decision? To answer them we build GroundKern, a testbed that separates evidence (a typed interaction graph and retrieved measurement traces), proposals (an LLM that emits a configuration with citations and an explanation) and enforcement (a runtime that validates, vetoes or stages the proposal). Because the evidence is an explicit, frozen object, it can be given to a non-LLM selector, ablated, or partially deleted; because enforcement is separate, its errors can be counted against labels computed from raw measurements.

Auditing ten LLMs, from 7B open-weight to hosted frontier models, on real Linux timer-slack and CPU-affinity interventions, we find that the three promises come apart. Evidence is necessary: without it, nine models mostly choose a worse configuration, and the local models we audited justify that choice by inventing meanings for opaque configuration names. It is also sufficient: with it, nine models reproduce the choice of a selector that simply reads the training table, a choice that is also best on the test grid. Citations are valid but one-sided and front-loaded: 252 of 260 cited retrieval runs belong to the chosen configuration, and 209 come from the first half of the evidence list. Explanations are cited but not correct (Figure 1): 16 of 28 audited claims from local models and 25 of 69 from hosted models misreport values, ranges or units, attribute a value to the wrong source, or overstate a regularity, and most of these accompany a correct decision. Citation-specific reliance under deletion is rare and unstable across precision and evidence format.

Contributions. (i) An audit protocol for evidence-grounded LLM advice that combines a matched-information non-LLM baseline, evidence-source ablations, cited versus matched uncited deletion, claim-level verification of explanations against the supplied evidence, and outcome labels independent of enforcement, with all data roles frozen before test data are opened (§4). (ii) An audit of ten LLMs reported with every denominator, including an analysis of what models cite, a claim-level taxonomy of explanation errors, and tests of attribution judges, prompt robustness, mitigations and reasoning (§5). (iii) The GroundKern testbed, whose calibrated risk gate gives a cautionary result on selective prediction under shift, with all data and code as supplementary material (§3).

The task is deliberately small, one workload family on one host, and its default configuration is best, so the audit can reveal failures of added value but not successes; we make no LLM-superiority, deployment-efficacy or production-safety claim (see Limitations).

2 Related Work

Our audit examines a cited rationale at four levels: the decision, the citations, the claims made about the cited evidence, and the reliance of the decision on that evidence. We relate each level to prior work, then turn to LLMs as decision makers and to risk control.

Citation quality and attribution. Retrieval-augmented generation conditions outputs on retrieved documents (Lewis et al., 2020), and attribution research asks whether the resulting text is supported by what it cites. AIS defines attribution to identified sources (Rashkin et al., 2023); Attributed QA and ALCE benchmark attribution and citation quality (Bohnet et al., 2022; Gao et al., 2023b); RARR revises text to restore attribution (Gao et al., 2023a); and generative search engines leave many statements unsupported by the sources they cite (Liu et al., 2023). These studies ask whether each citation supports its statement. We additionally ask which evidence gets cited at all, and find citations that are valid yet one-sided and concentrated at the start of the evidence list (§5.2).

Claim-level accuracy against data. FActScore checks atomic facts against a knowledge source (Min et al., 2023). Data-to-text generation has long documented factual errors in text produced from tables (Wiseman et al., 2017); Thomson and Reiter (2020) annotate every error in generated basketball summaries against the source data, including incorrect numbers, names and words, and summarization work distinguishes intrinsic from extrinsic hallucination (Maynez et al., 2020; Ji et al., 2023). Statements requiring comparison, superlatives or aggregation over a table are hard both to verify and to generate faithfully (Chen et al., 2020b,a). We apply this error-annotation practice to the free-text rationales

of instruction-tuned LLMs, where the supplied evidence is small enough for every claim to be checked exactly, and find the same superlative, range and quantifier errors (§5.3).

Faithfulness and reliance. Faithfulness must be evaluated separately from plausibility (Jacovi and Goldberg, 2020). ERASER measures the comprehensiveness and sufficiency of rationales by removing them (DeYoung et al., 2020); counterfactual and reconstruction tests probe natural-language explanations (Atanasova et al., 2023); and chain-of-thought explanations can omit the factors that drive a prediction (Turpin et al., 2023) or be tested by intervening on the reasoning (Lanham et al., 2023). Self-explanation faithfulness varies across models and tasks (Madsen et al., 2024), and many such tests measure output-level self-consistency rather than faithfulness to internal computation (Parcalabescu and Frank, 2024); ContextCite attributes generations to context by ablation (Cohen-Wang et al., 2024). Our matched deletion is an ERASER-style comprehensiveness test with a same-kind uncited control and, in the terms of Parcalabescu and Frank (2024), a self-consistency test (§5.4).

How models use supplied context. Models may ignore context that conflicts with parametric knowledge (Longpre et al., 2021) or accept coherent counter-evidence while favoring their priors (Xie et al., 2024); irrelevant context distracts them (Shi et al., 2023), and information in the middle of long contexts is used less (Liu et al., 2024). Our evidence ablation examines the complementary case in which the context is decisive and the prior is misleading (§5.1), and our citation analysis adds a position effect in what models choose to cite.

LLMs as decision makers. LLMs can optimize by proposing candidates from past evaluations (Yang et al., 2024), and configuration advisors mine tuning hints from manuals (Trummer, 2022), prune Bayesian-optimization searches with LLM-extracted knowledge (Lao et al., 2024), generate complete configurations (Giannakouris and Trummer, 2025) or tune Linux kernel build options as agents (Chen et al., 2024; Lin et al., 2025); classical tuners use Gaussian processes, reinforcement learning or black-box optimization (Van Aken et al., 2017; Zhang et al., 2019; Herodotou et al., 2021; Snoek et al., 2012; Golovin et al., 2017; Lee et al., 2022). These works score the chosen action end to end, whereas we ask what the model adds over a selector given the same evidence and whether its stated reasons are correct; we do not compare with these systems head to head. Following arguments against random or standard splits (Gorman and Bedrick, 2019; Søgaard et al., 2021) and for reporting matched budgets (Dodge et al., 2019), our data roles come from disjoint operating conditions and every baseline has an explicit budget.

Uncertainty and risk control. Self-consistency aggregates sampled outputs (Wang et al., 2023); LLMs can partly predict their own correctness (Kadavath et al., 2022), verbalized confidence can be better calibrated than token probabilities for RLHF models (Tian et al., 2023) yet remains overconfident (Xiong et al., 2024), and semantic entropy measures uncertainty over meanings (Kuhn et al., 2023). Selective classification abstains to meet a target risk (Geifman and El-Yaniv, 2017), and conformal methods give guarantees for language-model outputs (Quach et al., 2024), their factuality (Mohri and Hashimoto, 2024) and monotone losses (Angelopoulos et al., 2024), with extensions for non-exchangeable or drifting data (Farinhas et al., 2024; Barber et al., 2023; Gibbs and Candès, 2021); none implies that a window reset after a detected change (Bifet and Gavaldà, 2007) restores a fixed risk level. Our rank gate is a restricted class-conditional instance of this family, and we report where its assumptions fail (§5.5).

3 Testbed: GroundKern

GroundKern is an evidence-grounded generation pipeline for system configuration that we use as an instrument (Figure 2). Four properties matter for the audit. The evidence is *addressable* (every citation resolves to one item), *portable* (a non-LLM selector can receive the same object) and *editable* (kinds or items can be removed), and enforcement is *independent* of generation, so its errors can be counted against labels the generator never sees.

Evidence as a structured context. The store holds typed items, each with a stable ID, a kind and a provenance record: summary tables, individual measurement traces, and edges of a knob-interaction graph. The graph is a small knowledge representation with a strict admission rule: for a lower-is-better loss L and baseline values a_0, b_0, an edge requires the contrast $I_{ab} = L(a,b) - L(a,b_0) - L(a_0,b) + L(a_0,b_0)$ from independently measured configurations, with a bootstrap interval

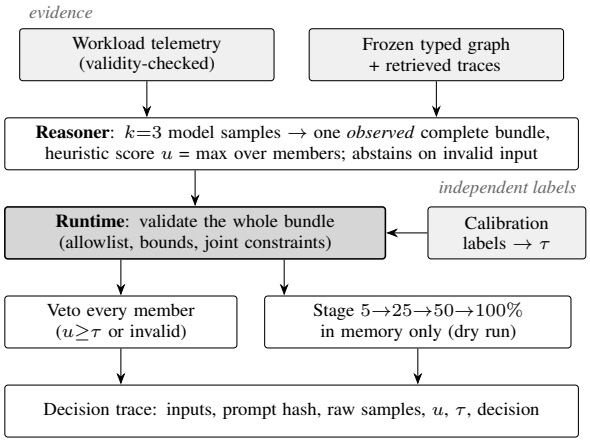


Figure 2: GroundKern decision path. Evidence, generation and enforcement are separate components: the frozen evidence object can be given to a non-LLM selector, ablated or partially deleted, citations can be resolved against its item IDs, and gate errors can be counted against independently defined labels. The REST runtime is a dry run; a separate per-thread actuator applies, verifies and restores timer slack and affinity in owned child processes.

over runs that excludes zero, and records its scope, values, measurement IDs and interval. Edges describe empirical interactions, not causal couplings (Appendix A). For the audit, the evidence is frozen and serialized as a JSON list that is identical for every advisor.

Generation. The reasoner retrieves typed neighbors and lexically matched trace text (Lewis et al., 2020) and asks a model for a complete proposal, a set of knob values, with cited item IDs and an explanation. In deployment it draws $k = 3$ samples (Wang et al., 2023), returns a modal complete proposal actually present in a response, scores each member by sample disagreement u (a heuristic; Appendix A), and abstains when evidence would be truncated. The audit (§4) instead calls each model directly on the frozen evidence, one response per seed.

Enforcement. The runtime treats a proposal as a unit: it validates every member (allowlisted knobs, bounds, joint constraints, no duplicates) and, if any member score satisfies $u \geq \tau$, vetoes all of them; otherwise the proposal enters a staged rollout. The REST runtime is a dry run that never writes kernel settings (Appendix A). The threshold τ comes from a rank gate, a selective-prediction rule (Geifman and El-Yaniv, 2017) calibrated on unsafe examples only, where $Y = 1$ marks an unsafe outcome defined from raw observations, never from whether a veto or rollback occurred. If unsafe calibration and test scores are exchangeable, the gate accepts an unsafe action with probability at most α (Proposition 1, Appendix A); this bounds misses among unsafe actions, not the unsafe fraction among accepted ones. The proof, drift and the per-thread actuator of §5.5 are in Appendix A.

Table 1: Audit levels, in the order of §5; all use the same frozen evidence.

Level	Manipulation	Measure
Decision	matched non-LLM selector; evidence ablation	held-out P95 change vs. control
Citation	— (observational)	kind, configuration, position
Claim	— (observational)	labels against the evidence
Reliance	cited vs. matched uncited deletion	decision change

4 Audit Protocol

The protocol treats configuration advice as a grounded generation task and audits its output at four levels, each with its own manipulation and measure (Table 1).

Task. The input is a list of four allowed configuration labels and a JSON evidence list; the output is one label, up to six cited evidence IDs and a free-text explanation. The labels name per-thread settings: timer slack of 50 μs or 1 ms crossed with affinity to all four CPUs or to CPU 0 (`s50000_aall`, `s50000_aone`, `s1000000_aall`, `s1000000_aone`). They are opaque by design: nothing in a label says which setting is better. The objective is the 95th-percentile (P95) wake-up lateness; `s50000_aall` is the control. Runs execute in fresh child processes on one Linux host (Appendix B).

Data roles and frozen evidence. Training, retrieval, calibration and test data come from disjoint wake-up periods of 2, 3, 4 and $\{5, 8\}$ ms (Søgaard et al., 2021). Each period has ten randomized blocks of the four configurations with 40 recorded events per run: 200 runs and 8,000 events, collected under a plan saved in advance. The evidence contains three kinds of items: the training table (mean training-run P95 per configuration), an interaction-graph item (the training contrast, $-18.6\,\mu$s, whose prespecified interval includes zero, so the graph has no edge) and 40 retrieval items, one per retrieval

run, listed in block order. Selections and thresholds are frozen, with hashes, before calibration and test data are opened; test outcomes never enter a prompt.

Decisions. The equally informed baseline is a table selector that picks the minimum of the training table; a saturated two-factor model of the same means is equivalent. It uses the same training runs as the LLMs and no online trials; matched-budget bandit, RL and BO optimizers (Auer et al., 2002; Snoek et al., 2012) are further references (Appendix B). Ten LLMs receive the same system prompt, evidence and output schema (Appendix C) at temperature 0 with five seeds: Qwen2.5 7B and 14B (Qwen Team, 2024), Llama-3.1-8B (Grattafiori et al., 2024) and Phi-4 (Abdin et al., 2024) locally as hashed 4-bit GGUF files in llama.cpp[1], and Llama-3.3-70B, Qwen3-235B-A22B (Yang et al., 2025), DeepSeek-V3.2 (DeepSeek-AI, 2025), Nemotron-3-Ultra-550B (NVIDIA, 2026), Gemini 3.8 Flash (Google DeepMind, 2026b) and Gemini 3.1 Pro preview (Google DeepMind, 2026a) through OpenRouter[2], whose weights cannot be hashed. Four input variants isolate the evidence: full, no graph, no retrieval, and no evidence. Each choice is scored by paired replay on the held-out grid: its run P95 in each test block and its change relative to the control in the same block, with a Student-t interval over ten blocks.

Citations. For every valid full-context answer we record the kind of each cited item, whether a cited run belongs to the chosen or to a competing configuration, and its position in the evidence list. Since the schema admits only valid IDs, these counts describe which evidence a model presents.

Claims. Following error annotation in data-to-text evaluation (Thomson and Reiter, 2020), every quantitative or comparative claim in each distinct explanation is checked against the prompt evidence and labeled *correct*, *overstated*, *wrong* (including partly wrong), *wrong unit*, *wrong attribution* (a value assigned to the wrong source) or, for no-evidence prompts, *unsupported prior*. A claim is *misstated* if it is wrong, misattributed, overstated or in the wrong unit; the single partly wrong claim is reported but not counted, which makes the counts conservative. A list-valued statement counts as one claim and a comparison as another; "consistently" or "always" lowest is overstated, since the chosen configuration is lowest in 8 of 10 retrieval blocks. Local models returned byte-identical text across seeds, so each distinct explanation is audited once per variant; hosted models varied across seeds and providers, so all distinct full-context explanations are audited. DeepSeek-V3.2's providers ignored the 400-character limit in 14 of 20 answers; these are audited in full. An AI assistant drafted the labels under these rules and one author verified all, changing none; two blind non-author annotators agreed ($\kappa = 0.92, 0.86$), and the three-way majority equals every verified label (App. C).

Reliance. Following the comprehensiveness test of ERASER (DeYoung et al., 2020), for each seed we delete the items cited in the full-context answer and, separately, the same number of uncited items of the same kinds. A decision that changes only under cited deletion indicates citation-specific reliance. A seed is non-estimable when no same-kind control exists, as for the training table, the only item of its kind.

5 Results

We report the audit at four levels, from the decision to the text that justifies it: what models choose (§5.1), what they cite (§5.2), what their explanations claim (§5.3) and whether the cited evidence matters (§5.4). A final subsection evaluates the testbed's risk gate (§5.5).

5.1 Decisions: Evidence Is Necessary and Sufficient

The table selector and its saturated-model equivalent choose the control, which is also best on the test grid, so any other choice is a loss; control test P95 is 179.7 ± 102.0 and 155.7 ± 64.4 μs at 5 and 8 ms (95% half-widths across ten blocks). Matched-budget random search, Q-learning, UCB1 and BO converge on it as well, choosing it in 98–99% of 1,000 seeds at $B = 40$ (Appendix B).

Table 2 gives each model's choice under every input condition. With evidence, nine of ten models choose the control in all three evidence variants, exactly as table lookup does; Llama-3.1-8B instead picks `s50000_aone`, which is worse on replay ($-145 \pm 125\%$ and $-180 \pm 156\%$). Removing the graph item changes no choice, and removing the retrieval runs changes none either: the train-

[1] https://github.com/ggml-org/llama.cpp, via llama-cpp-python 0.3.35.

[2] https://openrouter.ai

Table 2: LLM decisions with evidence (full, no graph, no retrieval), without it, and after deleting cited or matched uncited evidence; superscripts count seeds; n/e: non-estimable; †hosted. Paired test P95 change versus control at 5 and 8 ms (%, 95% half-width): ctrl 0; 50/1 -145 ± 125 and -180 ± 156; 1m/all -599 ± 188 and -631 ± 179; 1m/1 -631 ± 183 and -699 ± 206.

Model	Evidence	None	Del. cited	Del. unc.
Qwen-7B	ctrl	50/1	n/e	n/e
Llama-8B	50/1	50/1	1m/all	ctrl
Qwen-14B	ctrl	50/1	n/e	n/e
Phi-4	ctrl	1m/1	50/1	ctrl
Llama-70B†	ctrl	1m/1	ctrl	ctrl
Qwen3-235B†	ctrl	1m/1[3] ctrl 50/1	ctrl	ctrl
DeepSeek-V3.2†	ctrl	50/1	n/e	n/e
Nemotron-550B†	ctrl	ctrl	ctrl	ctrl
Gemini-Flash†	ctrl	50/1	n/e	n/e
Gemini-Pro†	ctrl	50/1	n/e	n/e

ing table alone suffices. Without evidence, every model except Nemotron, which declares the default, chooses a worse setting in at least four of five seeds, usually single-CPU affinity with 50 μs slack (six models) or with 1 ms slack (three). The evidence is therefore necessary; in this task it is also sufficient, since no model improves on the selector that simply reads it. Sufficiency holds only while the evidence agrees with itself. On further kernel instances, choices follow the evidence even under permuted labels, but Llama-3.3-70B leaves the table in all seeds when 4 ms runs compete with it (Appendix J). In a database task, 21 of 27 hosted answers follow competing runs rather than the table, toward a setting that is not better on test (Appendix K).

5.2 Citations: Valid, One-Sided and Front-Loaded

Every citation refers to a supplied item, since the schema permits nothing else, but what models choose to cite is far from a balanced account of the evidence. Across the 50 full-context answers, 285 citations comprise 25 of the training table, none of the interaction-graph item, and 260 of individual retrieval runs. Of these 260, 252 are runs of the configuration the model chose; only Gemini 3.1 Pro ever cites a run of a competing configuration. The cited evidence is thus confirmatory: no answer from the other nine models cites a run of a competing configuration, although `s50000_aone` beats the control in two of ten retrieval blocks.

Citations also favor the start of the evidence list. Retrieval runs appear in block order, and 209 of the 260 retrieval citations fall in the first five of ten blocks. Seven models cite exactly a prefix of the blocks (typically 0–4 or 0–5) and so never cite the chosen configuration's lowest run. Rerunning the hosted models (Appendix E) shows that, under the cap, the prefix follows list position (Liu et al., 2024): with the runs reversed, four of six models cite those listed first; uncapped, models cite 9.0 runs, all of the chosen configuration.

Table 3: Verification of quantitative and comparative claims in explanations against the supplied evidence (AI-drafted labels, verified by an author). Corr.: correct; Over.: overstated; Wrong: wrong or partly wrong value; Unit/Src: wrong unit or evidence source; Prior: unsupported prior without evidence; Misst.: wrong, misattributed, overstated or wrong unit (the one partly wrong claim is not counted). Generated by `claim_taxonomy.py`.

Model	n	Corr.	Over.	Wrong	Unit/Src	Prior	Misst.
Qwen-7B	5	1	1	1	1	1	3
Llama-8B	7	–	–	6	–	1	5
Qwen-14B	6	1	1	–	3	1	4
Phi-4	10	5	3	1	–	1	4
Local, all variants	*28*	*7*	*5*	*8*	*4*	*4*	*16*
Llama-70B	7	5	–	2	–	–	2
Qwen3-235B	6	3	3	–	–	–	3
DeepSeek-V3.2	16	10	2	4	–	–	6
Nemotron-550B	23	13	5	5	–	–	10
Gemini-Flash	12	10	2	–	–	–	2
Gemini-Pro	5	3	2	–	–	–	2
Hosted, full context	*69*	*44*	*14*	*11*	–	–	*25*

Table 4: Examples of misstated claims (paraphrased as audited) and the check against the supplied evidence; all latencies in μs.

Model, label	Claim	Evidence
Gemini-Pro†, overstated	control "consistently the lowest in retrieval runs"	lowest in 8 of 10 blocks
Nemotron†, wrong	`s50000_aone` ranges from 85.0 to 430.7	maximum is 571.84
DeepSeek†, wrong	control "consistently lower than any" `s50000_aone` run	ranges overlap: $85.033 < 183.217$
Qwen-7B, wrong source	training table gives control 126.36	126.36 is one retrieval run; training mean 132.94
Qwen-14B, wrong unit	training table gives control 132.9413 ms	evidence is in μs
Qwen-14B, prior	"50000" denotes a smaller dataset size	no evidence given; name is opaque

5.3 Explanations: Cited but Not Correct

The explanations are often wrong about what the cited items say (Table 3): 16 of 28 audited claims from local models and 25 of 69 from hosted models are misstated; with either blind annotator's labels instead, 37 or 38 of 97 are. Errors fall into four

recurring patterns (Table 4).

Overstated regularities. Thirteen of the 19 overstatements assert that the chosen configuration is lowest "consistently" or in "all" retrieval runs, whereas it is lowest in 8 of 10 retrieval blocks; the other six claim "significant" differences that no test in the evidence supports. These claims are easier to make when, as above, the cited runs are all of one configuration.

Range and extremum errors. All 11 wrong values from hosted models misreport a range, an extremum or a cross-configuration ordering. Nemotron repeatedly gives the `s50000_aone` range as 85.0–430.7 μs (its maximum is 571.84), and DeepSeek-V3.2 states that the control is "consistently lower than any" `s50000_aone` run although the ranges overlap (Figure 1). Llama-3.1-8B reports 571.84 μs, the *maximum* of `s50000_aone`, as the lowest P95, consistent with its wrong choice.

Source and unit confusion. Qwen2.5-7B attributes the retrieval value 126.36 to the training table, whose mean is 132.94; Qwen2.5-14B reports the training mean 132.9413 in milliseconds in three explanations, although the evidence declares microseconds.

Invented semantics. Without evidence, all four local models justify their choice by assigning meanings to opaque identifiers, for instance reading "50000" as a smaller dataset size or "one" as a more aggressive approach. The configuration names carry no such information.

These errors are largely decoupled from the decision: 35 of the 41 misstated claims accompany a choice of the control. In the database task the coupling reverses: nine explanations misreport the training table as agreeing with a choice it contradicts (Appendix K). Wording also varies without the decision: hosted models gave two to five distinct explanations each while always choosing the control. Valid citations thus establish provenance, and a correct decision can coexist with an incorrect account of the evidence that supports it.

Mitigations help but do not certify. For the hosted models, an instruction to verify every stated number cuts flagged quantifiers from 21 to 1 of 30 explanations and range errors from 7 to 2, and precomputed summaries remove range errors, whereas enabling internal reasoning leaves the overstatement in place; reading the explanations still finds false comparisons the automatic checks miss, and these answers lack verified labels (Appendices F and G).

5.4 Reliance: Rarely Specific, Rarely Stable

Five models cite the single training table, so matched deletion is non-estimable for them. Among the others, only Phi-4 changes its choice under cited but not uncited deletion (all five seeds), to a choice that contradicts the remaining table, and only at 4 bits (Appendix H); Llama-3.1-8B changes under both deletions, and Llama-3.3-70B, Qwen3 and Nemotron under neither. A redesign with a split table and configuration-matched controls is estimable for most models (Appendix I): no hosted model then changes, and only bf16 Llama-3.1-8B switches under cited deletion alone.

5.5 Enforcement: A Calibrated Gate under Shift

A test run is *unsafe* when more than 10% of its 40 events exceed a predeclared 200 μs deadline, a label computed from timing alone. For the gate, the score is a configuration's training deadline-miss fraction, fixed before calibration on 4 ms runs only; all three prespecified $\alpha \in \{.05, .10, .20\}$ yield $\tau = .9225$ because the unsafe-score order statistics are tied. Every candidate is executed in an isolated process, so accepted and vetoed candidates both have observed outcomes (Figure 3 in Appendix B).

Unsafe acceptance is $5/25 = 20.0\%$ and $4/24 = 16.7\%$ among unsafe runs, significantly above the 5% level for this calibration draw (one-sided binomial $p = .007$ and .03) but within sampling error of 10% ($p = .10, .21$); unsafe fractions among accepted runs are $5/20$ and $4/20$, and veto precision is $20/20$ in each period. As with LLM explanations, a nominally calibrated signal is only as good as the exchangeability it assumes, and a period shift can be enough to break it. The bounded actuator restored both original settings in all 80 staged executions (Appendix B).

6 Discussion

Decisions and rationales come apart. Most misstatements accompany a correct choice. This pattern is consistent with a decision driven by a simple reading of the evidence, here the minimum of the training table, and a rationale that is generated as a separate summary of the retrieval runs. Choosing the smallest of four means is easy; summarizing forty noisy measurements with ranges, extremes and quantifiers is where the errors concentrate. Because the decision is right, an end-to-end evaluation would never surface these errors, and a reader who

trusts the rationale could come away with a false picture of the evidence, for instance that the chosen configuration is never beaten. This mirrors findings that explanations can be plausible yet unfaithful to what drives a prediction (Jacovi and Goldberg, 2020; Turpin et al., 2023), with the difference that here the error is checkable against the prompt itself.

Citations as selective presentation. A citation list is also an editorial choice. Nine of ten models cited only runs of the configuration they chose, and most cited the first runs in the list, so a reader who follows the citations sees supporting evidence only and never the blocks in which a competitor wins. It also licenses overstatement (§5.3): "consistently the lowest" cannot be contradicted by the cited runs, only by those left uncited. The six-citation cap explains the preference for early items, which follows list position, but not the preference for confirming ones, which persists without it (Appendix E).

Scale reduces but does not remove misstatement. Every audited model misstated at least two claims, and "consistently the lowest" recurs across families. On full-context prompts, misstatement falls with size among open-weight models (7/8 claims at 7–15B, 21/52 at 70–671B; Spearman $\rho = -0.78$ over eight models, a trend on few claims; Appendix D) yet stays at 17–50% per hosted model; unit and source errors occur only at small scale, overstatements and wrong values at every scale. The superlative, range and quantifier errors long documented in data-to-text generation (Thomson and Reiter, 2020; Chen et al., 2020a) are therefore not removed by instruction tuning or scale in the models we tested.

Implications for attribution metrics. Attribution benchmarks ask whether each cited passage supports its statement, often with an entailment judge (Rashkin et al., 2023; Gao et al., 2023b). One-sided citation lists are invisible to such per-citation scores, and many misstated claims quote correct numbers and err only in a comparison or quantifier, near-misses that eight judges, including the fact-checkers AlignScore and MiniCheck, fail to separate from correct claims (balanced accuracy 0.47–0.68; Appendix C). Evaluations should therefore also measure what is left out, such as the share of contrary evidence cited, and check claims exactly where possible.

What deletion can and cannot show. Matched deletion measures whether a decision is sensitive to what is cited, which is a form of self-consistency rather than faithfulness to internal computation (Parcalabescu and Frank, 2024). It is estimable only with redundant items of the same kind; half of our models cited the single training table, which has no control. Our redesign with matched alternatives for every citable item makes it estimable, yet controls still differ in which items remain.

When could an LLM add value? In the kernel task the evidence determines the best choice, so a selector that reads it is hard to beat. Priors could add value where measurements are noisy or conflict: in the database task, models without evidence mostly chose the test-best setting, yet with conflicting evidence they followed the runs (Appendix K). How to combine priors with evidence, and how to check priors rather than assume them, as the invented label meanings show, remains open.

7 Conclusion

A rationale that cites evidence promises a good decision, true claims about the evidence, and reliance on what it cites. Auditing them separately for LLM configuration advice, we found that they came apart. Ten LLMs needed the evidence to choose well but added nothing beyond a selector that reads it; their citations were valid but one-sided (252 of 260 cited runs back the chosen configuration); 41 of 97 audited claims misstated validly cited evidence, mostly alongside a correct decision; and citation-specific reliance under deletion was rare and unstable across precision and evidence format.

For NLP evaluation, these results suggest three practices. First, citation validity and passage-level support are not enough: an explanation can cite valid items, omit the contrary ones and still misreport what it cites, so attribution should be checked claim by claim and for what is left out. Second, the errors that data-to-text research documented for tables, namely wrong numbers, superlatives, ranges and overstated regularities, persist in the free-text rationales of instruction-tuned models of every size we tested, and deserve targeted checks. Third, LLM decision support should be compared with a baseline given the same evidence, reporting decision quality and rationale correctness separately.

Limitations

Task and generality. The main audit covers one grounded-advice task with four candidate labels, English prompts and a small numeric evidence set that determines the best choice, and the default is best on the test grid. It can therefore expose missing added value and misreported evidence, but it cannot show that an LLM adds value on harder tasks, and its error rates need not transfer to domains with textual evidence (manuals, logs, documentation), longer contexts or other languages. The configuration labels are opaque by design; with descriptive labels, models' priors could help or hurt differently. Three further evidence instances from the same host and workload family (Appendix J) vary the values, labels and session. Only the hosted models were rerun on them, and their explanations were scored by automatic checks, not verified labels. A second domain, SQLite settings measured for this paper (Appendix K), reproduces selective citation and misstatement. It has one workload and one machine and covers only the hosted models, and its explanations were read with an AI assistant rather than verified by annotators.

Annotation. Claim labels were drafted with an AI assistant and verified by one author; agreement with two independent blind annotators who are not authors is high (Cohen's $\kappa = 0.92$ and 0.86; Fleiss' $\kappa = 0.85$ with the verified labels), but rests on few annotators and a small sample (97 claims). The annotators labeled claims that had already been extracted, so the extraction step (which statements count as claims) was not independently validated, and only quantitative or comparative claims were audited. Segmenting explanations into claims and deciding when a claim is "overstated" involve judgment; we fixed both rules in advance (a list counts as one claim, a comparison as another; "consistently" is overstated when a configuration is lowest in 8 of 10 blocks), and supply every label with its check so that the annotation can be repeated or automated.

Models and prompting. The main audit uses one system prompt and output schema at temperature 0. A rerun of the hosted models with a paraphrased prompt, reordered evidence, no citation cap and sampling (Appendix E) leaves decisions and one-sided citation unchanged, but scores its explanations only with automatic proxies, not verified labels, as does the mitigation rerun (Appendix F), whose checks see only two error types; local models and evidence formatting were not varied, and reasoning was toggled only within four hosted models (Appendix G); the two Gemini models reasoned by default in the main audit. Local explanations were byte-identical across seeds, and five greedy seeds are not independent samples. Local models were 4-bit quantized; reruns of Llama-3.1-8B and Phi-4 with bf16 weights and of Qwen2.5-7B at 8 bits reproduce the decisions in all evidence and no-evidence variants and the explanation errors recur, but Phi-4 no longer changes its choice under cited deletion (Appendix H); Qwen2.5-14B could not be rerun at full precision. Hosted models were served by several providers, could not be hashed, varied across seeds and are not reproducible bit for bit; DeepSeek-V3.2's providers ignored the explanation length limit. The six-citation cap and the 400-character limit shape what models cite and write; the robustness rerun shows that the citation prefix depends on the cap.

Reliance. Matched deletion measures the sensitivity of a decision to cited evidence, a form of self-consistency, not faithfulness to internal computation. In the main design it was non-estimable for half of the models, because they cited the only item of its kind; the redesign of Appendix I is estimable for most, but its controls still differ in which blocks remain, and splitting the table changed one model's choice.

Effect on readers. We show that rationales misstate checkable facts, not that readers are misled by them; no human-subject study was run. The supplement includes a between-subjects protocol prepared for pre-registration (misstated, minimally corrected or no rationale; evidence probes with computed answers), its materials and analysis code, for future work.

Measurement scope. All measurements use one periodic workload family on one shared host, with data roles separated by period rather than by workload family or hardware; run-level intervals do not account for uncontrolled interference or correlated system state and are unadjusted for multiple comparisons. Measured graph induction emits no edge, so the graph item is uninformative in this task. Selector, BO and RL comparisons replay recorded runs rather than executing closed loops, and because control is best they can reveal only losses. The telemetry prototype's scheduler and block-I/O

proxies are not end-to-end request latencies, and the local benchmarks in the appendix do not substitute for application workloads such as web serving, databases or GPU inference.

System and safety scope. The REST runtime is a dry run that never writes kernel settings, and the separate actuator changes only an owned child thread's timer slack and affinity; neither establishes traffic isolation, a deployed multi-knob actuator or recovery from irreversible effects. The reasoner awaits model samples synchronously, and its logs are append-only files without tamper resistance. Prompting a model to respect graph edges does not enforce them; only the runtime's explicit constraints are enforced. The rank gate's assumptions fail under drift, selective labeling or a score updated with calibration data, as the period-shift result shows, and the gate replay neither verifies deployment assumptions nor establishes recovery. We therefore make no LLM-superiority, deployment-efficacy, no-worse-than-default or production-safety claim; Section 4 and Appendix A describe the comparisons and engineering such claims would require.

Ethics Statement

Intended use and risks of the findings. The paper studies how far users can trust LLM rationales that cite supplied evidence. Its main risk is misreading in either direction: treating valid citations or a correct final choice as evidence that a rationale is accurate, which our results argue against, or reading the audit as evidence that LLM configuration advisors are safe or effective, which it does not show (see Limitations). We recommend that LLM advice affecting real systems be paired with claim-level checks of its rationale and with enforcement that does not rely on the model.

Data, annotation and models. The data are timing measurements of local software workloads, synthetic configuration labels, and model responses to one fixed technical prompt; they contain no personal data and involve no human participants. The explanation-claim labels were drafted with an AI assistant and verified by an author (see Use of AI assistants); two further annotators, fellow researchers who are not authors, volunteered without payment to label them blind and independently for an agreement study and consented to the release of the labels. All models were used for research inference under their licenses: Apache 2.0 (Qwen), the Llama 3.1 and 3.3 Community Licenses, MIT (Phi-4, DeepSeek-V3.2), OpenMDW-1.1 (Nemotron) and the Gemini API terms. Code, measurements, prompts, responses and annotations are provided as supplementary material and will be released publicly under Apache 2.0; the stored responses remain subject to each model's terms.

System safety. The REST runtime performs no kernel writes; the experimental adapter changes only its owned child thread's timer slack and affinity, restores both settings, and performs no host-wide writes. Broader interventions require an isolated testbed, documented rollback limits and appropriate authorization.

Compute. The study used about 3.2 CPU-hours of local inference and CPU-only attribution judges without a GPU, about 855 hosted API calls (150 in the main audit, about 0.5 USD; 152 in the robustness rerun, 0.83 USD; 95 in the mitigation rerun, 0.70 USD; 40 in the reasoning toggles, 0.18 USD; about 60 in the bf16 reruns; about 200 in the redesigned reliance test, 0.64 USD; 90 on further evidence instances, 0.45 USD; 65 in the database task, 0.28 USD), and seconds of kernel measurement (Appendix C).

Use of AI assistants. The research idea, problem formulation and study design are the authors'. AI assistants were used for writing and debugging code, for drafting and restructuring prose, for locating references and for drafting the explanation-claim labels, which an author then verified; every reference was checked against its publisher, ACL Anthology, Crossref, proceedings or arXiv record, and every claim label is released with its check so that it can be verified. The authors take full responsibility for the content. Every reported number is produced by the supplied scripts from raw logs whose SHA-256 hashes are recorded, so that it can be recomputed independently; no generated value substitutes for an unavailable observation.

References

Marah Abdin, Jyoti Aneja, Harkirat Behl, et al. 2024. Phi-4 technical report. arXiv:2412.08905. https://arxiv.org/abs/2412.08905.

Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. 2024. Conformal

risk control. In *International Conference on Learning Representations*. https://arxiv.org/abs/2208.02814.

Pepa Atanasova, Oana-Maria Camburu, Christina Lioma, Thomas Lukasiewicz, Jakob Grue Simonsen, and Isabelle Augenstein. 2023. Faithfulness tests for natural language explanations. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 283–294. https://aclanthology.org/2023.acl-short.25/.

Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256. https://doi.org/10.1023/A:1013689704352.

Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. 2023. Conformal prediction beyond exchangeability. *The Annals of Statistics*, 51(2):816–845. https://doi.org/10.1214/23-AOS2276.

Albert Bifet and Ricard Gavaldà. 2007. Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining*, pages 443–448. https://doi.org/10.1137/1.9781611972771.42.

Bernd Bohnet, Vinh Q. Tran, Pat Verga, Roee Aharoni, Daniel Andor, Livio Baldini Soares, Massimiliano Ciaramita, Jacob Eisenstein, et al. 2022. Attributed question answering: Evaluation and modeling for attributed large language models. arXiv:2212.08037. https://arxiv.org/abs/2212.08037.

Huilai Chen, Yuanbo Wen, Limin Cheng, Shouxu Kuang, et al. 2024. AutoOS: Make your OS more powerful by exploiting large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 7511–7525. https://proceedings.mlr.press/v235/chen24at.html.

Wenhu Chen, Jianshu Chen, Yu Su, Zhiyu Chen, and William Yang Wang. 2020a. Logical natural language generation from open-domain tables. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7929–7942. https://aclanthology.org/2020.acl-main.708/.

Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and William Yang Wang. 2020b. TabFact: A large-scale dataset for table-based fact verification. In *International Conference on Learning Representations*. https://arxiv.org/abs/1909.02164.

Benjamin Cohen-Wang, Harshay Shah, Kristian Georgiev, and Aleksander Mądry. 2024. ContextCite: Attributing model generation to context. In *Advances in Neural Information Processing Systems*, volume 37. https://arxiv.org/abs/2409.00729.

DeepSeek-AI. 2025. DeepSeek-V3.2: Pushing the frontier of open large language models. arXiv:2512.02556. https://arxiv.org/abs/2512.02556.

Jay DeYoung, Sarthak Jain, Nazneen Fatema Rajani, Eric Lehman, Caiming Xiong, Richard Socher, and Byron C. Wallace. 2020. ERASER: A benchmark to evaluate rationalized NLP models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4443–4458. https://aclanthology.org/2020.acl-main.408/.

Jesse Dodge, Suchin Gururangan, Dallas Card, Roy Schwartz, and Noah A. Smith. 2019. Show your work: Improved reporting of experimental results. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2185–2194. https://aclanthology.org/D19-1224/.

António Farinhas, Chrysoula Zerva, Dennis Ulmer, and André F. T. Martins. 2024. Non-exchangeable conformal risk control. In *International Conference on Learning Representations*. https://arxiv.org/abs/2310.01262.

Luyu Gao, Zhuyun Dai, Panupong Pasupat, Anthony Chen, Arun Tejasvi Chaganty, Yicheng Fan, Vincent Y. Zhao, Ni Lao, Hongrae Lee, Da-Cheng Juan, and Kelvin Guu. 2023a. RARR: Researching and revising what language models say, using language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16477–16508. https://aclanthology.org/2023.acl-long.910/.

Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023b. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6465–6488. https://aclanthology.org/2023.emnlp-main.398/.

Yonatan Geifman and Ran El-Yaniv. 2017. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30. https://proceedings.neurips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html.

Victor Giannakouris and Immanuel Trummer. 2025. λ-Tune: Harnessing large language models for automated database system tuning. *Proceedings of the ACM on Management of Data*, 3(1). https://doi.org/10.1145/3709652.

Isaac Gibbs and Emmanuel Candès. 2021. Adaptive conformal inference under distribution shift. In *Advances in Neural Information Processing Systems*, volume 34, pages 1660–1672. https://proceedings.neurips.cc/paper/2021/hash/0d441de75945e5acbc865406fc9a2559-Abstract.html.

Daniel Golovin, Benjamin Solnik, Subhodeep Moitra, Greg Kochanski, John Karro, and D. Sculley. 2017. Google Vizier: A service for black-box optimization. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1487–1495. https://research.google/pubs/pub46180/.

Google DeepMind. 2026a. Gemini 3.1 Pro model card. Model card. https://deepmind.google/models/model-cards/gemini-3-1-pro/.

Google DeepMind. 2026b. Gemini 3.8 Flash model card. Model card. https://deepmind.google/models/model-cards/gemini-3-8-flash/.

Kyle Gorman and Steven Bedrick. 2019. We need to talk about standard splits. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2786–2791. https://aclanthology.org/P19-1267/.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 herd of models. arXiv:2407.21783. https://arxiv.org/abs/2407.21783.

Herodotos Herodotou, Yuxing Chen, and Jiaheng Lu. 2021. A survey on automatic parameter tuning for big data processing systems. *ACM Computing Surveys*, 53(2):43:1–43:37. https://doi.org/10.1145/3381027.

Alon Jacovi and Yoav Goldberg. 2020. Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness? In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4198–4205. https://aclanthology.org/2020.acl-main.386/.

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38. https://doi.org/10.1145/3571730.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, et al. 2022. Language models (mostly) know what they know. arXiv:2207.05221. https://arxiv.org/abs/2207.05221.

Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations*. https://arxiv.org/abs/2302.09664.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, et al. 2023. Measuring faithfulness in chain-of-thought reasoning. arXiv:2307.13702. https://arxiv.org/abs/2307.13702.

Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2024. GPTuner: A manual-reading database tuning system via GPT-guided Bayesian optimization. *Proceedings of the VLDB Endowment*, 17(8):1939–1952. https://doi.org/10.14778/3659437.3659449.

Hyeonmyeong Lee, Sungmin Jung, and Heeseung Jo. 2022. STUN: Reinforcement-learning-based optimization of kernel scheduler parameters for static workload performance. *Applied Sciences*, 12(14):7072. https://doi.org/10.3390/app12147072.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

Hongyu Lin, Yuchen Li, Haoran Luo, et al. 2025. TuneAgent: Agentic operating system kernel tuning with reinforcement learning. arXiv:2508.12551. First released as OS-R1. https://arxiv.org/abs/2508.12551.

Linux man-pages project. 2026a. PR_SET_TIMERSLACK(2const). https://man7.org/linux/man-pages/man2/PR_SET_TIMERSLACK.2const.html.

Linux man-pages project. 2026b. sched_setaffinity(2). https://man7.org/linux/man-pages/man2/sched_setaffinity.2.html.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173. https://aclanthology.org/2024.tacl-1.9/.

Nelson F. Liu, Tianyi Zhang, and Percy Liang. 2023. Evaluating verifiability in generative search engines. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 7001–7025. https://aclanthology.org/2023.findings-emnlp.467/.

Shayne Longpre, Kartik Perisetla, Anthony Chen, Nikhil Ramesh, Chris DuBois, and Sameer Singh. 2021. Entity-based knowledge conflicts in question answering. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7052–7063. https://aclanthology.org/2021.emnlp-main.565/.

Andreas Madsen, Sarath Chandar, and Siva Reddy. 2024. Are self-explanations from large language models faithful? In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 295–337. https://aclanthology.org/2024.findings-acl.19/.

Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1906–1919. https://aclanthology.org/2020.acl-main.173/.

Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12076–12100. https://aclanthology.org/2023.emnlp-main.741/.

Christopher Mohri and Tatsunori Hashimoto. 2024. Language models with conformal factuality guarantees. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 36029–36047. https://proceedings.mlr.press/v235/mohri24a.html.

NVIDIA. 2026. Nemotron 3 Ultra: Open, efficient mixture-of-experts hybrid Mamba-transformer model for agentic reasoning. Technical report. https://research.nvidia.com/labs/nemotron/files/NVIDIA-Nemotron-3-Ultra-Technical-Report.pdf.

Letitia Parcalabescu and Anette Frank. 2024. On measuring faithfulness or self-consistency of natural language explanations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6048–6089. https://aclanthology.org/2024.acl-long.329/.

Victor Quach, Adam Fisch, Tal Schuster, Adam Yala, Jae Ho Sohn, Tommi S. Jaakkola, and Regina Barzilay. 2024. Conformal language modeling. In *International Conference on Learning Representations*. https://arxiv.org/abs/2306.10193.

Qwen Team. 2024. Qwen2.5 technical report. arXiv:2412.15115. https://arxiv.org/abs/2412.15115.

Hannah Rashkin, Vitaly Nikolaev, Matthew Lamm, Lora Aroyo, Michael Collins, Dipanjan Das, Slav Petrov, Gaurav Singh Tomar, Iulia Turc, and David Reitter. 2023. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4):777–840. https://aclanthology.org/2023.cl-4.2/.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. 2023. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *PMLR*, pages 31210–31227. https://proceedings.mlr.press/v202/shi23a.html.

Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. 2012. Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25. https://proceedings.neurips.cc/paper/2012/hash/05311655a15b75fab86956663e1819cd-Abstract.html.

Anders Søgaard, Sebastian Ebert, Jasmijn Bastings, and Katja Filippova. 2021. We need to talk about random splits. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 1823–1832. https://aclanthology.org/2021.eacl-main.156/.

Liyan Tang, Philippe Laban, and Greg Durrett. 2024. MiniCheck: Efficient fact-checking of LLMs on grounding documents. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8818–8847. https://aclanthology.org/2024.emnlp-main.499/.

Craig Thomson and Ehud Reiter. 2020. A gold standard methodology for evaluating accuracy in data-to-text systems. In *Proceedings of the 13th International Conference on Natural Language Generation*, pages 158–168. https://aclanthology.org/2020.inlg-1.22/.

Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher D. Manning. 2023. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5433–5442. https://aclanthology.org/2023.emnlp-main.330/.

Immanuel Trummer. 2022. DB-BERT: A database tuning tool that "reads the manual". In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*. https://doi.org/10.1145/3514221.3517843.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. 2023. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36. https://arxiv.org/abs/2305.04388.

Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic database management system tuning through large-scale machine learning. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*, pages 1009–1024. https://doi.org/10.1145/3035918.3064029.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Rep-*

resentations. https://arxiv.org/abs/2203.11171.

Sam Wiseman, Stuart M. Shieber, and Alexander M. Rush. 2017. Challenges in data-to-document generation. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 2253–2263. https://aclanthology.org/D17-1239/.

Jian Xie, Kai Zhang, Jiangjie Chen, Renze Lou, and Yu Su. 2024. Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts. In *International Conference on Learning Representations*. https://arxiv.org/abs/2305.13300.

Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. 2024. Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs. In *International Conference on Learning Representations*. https://arxiv.org/abs/2306.13063.

An Yang et al. 2025. Qwen3 technical report. arXiv:2505.09388. https://arxiv.org/abs/2505.09388.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *International Conference on Learning Representations*. https://arxiv.org/abs/2309.03409.

Yuheng Zha, Yichi Yang, Ruichen Li, and Zhiting Hu. 2023. AlignScore: Evaluating factual consistency with a unified alignment function. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11328–11348. https://aclanthology.org/2023.acl-long.634/.

Ji Zhang, Yu Liu, Ke Zhou, Guoliang Li, et al. 2019. An end-to-end automatic cloud database tuning system using deep reinforcement learning. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, pages 415–432. https://doi.org/10.1145/3299869.3300085.

A GroundKern Testbed Details

Design hypotheses. GroundKern is motivated by three hypotheses. First, a configuration's utility may depend on workload and hardware. Second, joint knob effects may be non-additive. Third, performance, failure probability and intervention cost may conflict. Each hypothesis requires measured interventions with explicit baselines.

For a loss L, define an interaction contrast from four independently measured configurations:

$$I_{ab} = L(a, b) - L(a, b_0) - L(a_0, b) + L(a_0, b_0).$$

For lower-is-better loss, negative I_{ab} indicates a beneficial non-additive interaction relative to the chosen baseline. A bootstrap interval must resample independent runs, not individual requests from a single run. The sign can change with workload, value range or kernel version, so an edge needs its scope, configuration values, measurement IDs and uncertainty interval. Such an edge describes an empirical interaction, not an unconditional causal coupling.

A DEPENDS-ON edge requires a validated admissibility rule or measured range conditioning; it cannot be inferred solely from the sign of I_{ab}. The seed graph is empty until edges pass this test. The principal question is whether a reasoner can use independently acquired evidence more effectively than an optimizer with the same inputs, after charging the cost of acquiring that evidence.

Restricted rank gate. The gate of §3 sets τ as follows; $Y = 1$ marks an unsafe outcome.

Proposition 1 (Restricted rank gate)**.** *Fix a score function before calibration. Suppose n unsafe calibration scores and the next unsafe test score are jointly exchangeable. Set $j = \lfloor \alpha(n+1) \rfloor$ for $0 < \alpha < 1$. If $j = 0$, veto all; otherwise let τ be the jth smallest unsafe calibration score and accept only when $u < \tau$. Then the unsafe-class acceptance probability, marginal over calibration and test scores, is at most $j/(n+1) \le \alpha$.*

Proof. By exchangeability the test score's rank among the $n + 1$ scores is uniform; it falls below the jth calibration score only in the first j ranks, and ties only reduce this. □

The bound controls misses among unsafe actions, $\Pr(A = 1 \mid Y = 1)$, not the unsafe fraction among accepted actions; a cost-selected threshold is capped at the rank threshold, and no unsafe calibration example implies veto-all. The gate accepts any fixed score; §5.5 evaluates it with a measured risk score rather than u.

Disagreement score. For a knob–value pair proposed in v of k samples, the reasoner's score is $u = 0.7(1-v/k)+0.3(1-w)$, clipped to $[0.02, 1]$, where w is the strongest retrieved synergy weight (zero when absent). It is neither predictive entropy nor a calibrated probability. Failed samples stay in the denominator, and repeated copies of a fallback are not counted as independent votes.

Bundle contract. A request contains a nonempty bundle identifier and a list of recommendations

Algorithm 1 Implemented dry-run bundle path

Require: Validated bundle B, fixed score function, calibration threshold τ

Validate every member and joint constraints before changing state
if another bundle is active **then**
  Return conflict
end if
if $\max_{a \in B} u(a) \geq \tau$ **then**
  Record veto for all members; return
end if
Stage the complete bundle in memory at 5%
for requested stage in $\{25, 50, 100\}\%$ **do**
  Advance the simulated bundle; record the transition
end for
On cancellation, clear the complete simulated bundle
Never infer an unsafe/safe training label from simulated completion

with `id`, `knob`, `proposed`, `uncertainty`, and optional rationale and provenance. Each request submits one bundle. Duplicate knobs, nonfinite scores, unsupported controls, and example-policy bound violations reject the entire request. Dirty-ratio changes must supply both values and satisfy the example policy that the background ratio is less than the foreground ratio. This is a declared policy, not proof that a configuration improves any workload. If any score satisfies $u \geq \tau$, all members are vetoed; otherwise all are staged together. A concurrent bundle receives a conflict response. Dry-run results return `applied=[]` and list simulated members separately.

Deployment requirements. A real actuator must snapshot all old values, validate the full target configuration and available host interfaces, serialize updates, verify readback, and restore every old value on failure. Multiple sysctl writes are not a kernel transaction: partial exposure must be measured and cannot be described as atomic hardware state. Canary shares require separate hosts or an appropriate isolation mechanism; changing a host-global sysctl cannot isolate a fraction of requests on that host. Missing, stale or malformed end-to-end telemetry must stop promotion. These capabilities are requirements, not features demonstrated by the dry-run state machine.

One illustrative decision trace. Consider supplied context with example values 5 and 15 for the two dirty ratios, and retrieved evidence IDs e_1, t_1. Suppose all three model samples agree on this pair and $w = 0$, giving $u = 0.30$. If an independently obtained threshold is $\tau = 0.25$, the whole bundle is vetoed; if $\tau = 0.35$, it can enter the dry-run canary after joint validation. The audit record includes the values, scores, threshold and decision. These numbers illustrate the contract and are not a measured recommendation or calibrated operating point. The initial artifact threshold is zero until calibration data is explicitly supplied.

Actuator. A separate adapter snapshots a child thread's timer slack and CPU affinity (Linux man-pages project, 2026a,b), validates the target, applies both settings, verifies readback and restores both originals in a `finally` path, including after partial application. Updates are sequential, not an atomic kernel transaction.

Expected cost of gated acceptance. Let $Y = 1$ denote an independently defined unsafe outcome and $A = 1$ acceptance. An outcome label is not defined by whether a rollback happened. For example, a benchmark can predeclare a request deadline, count observed failures or deadline exceedances, and define an action-level breach from these raw observations in a fixed window. Rejected actions have unknown counterfactual outcomes unless evaluated in an isolated experiment; they must not automatically be labeled safe or unsafe.

Theorem 1 (Cost identity)**.** *For costs* $c_{\mathrm{FN}}, c_{\mathrm{FP}}, \lambda \geq 0$*, define* $C = c_{\mathrm{FN}}\mathbf{1}\{Y = 1, A = 1\} + c_{\mathrm{FP}}\mathbf{1}\{Y = 0, A = 0\} + \lambda D$*, where* $D \geq 0$ *is measured exposure or SLO debt. Write* $\pi = \Pr(Y = 1)$*,* $q = \Pr(A = 1 \mid Y = 1)$ *and* $r = \Pr(A = 0 \mid Y = 0)$*. Then*

$$\mathbb{E}C = c_{\mathrm{FN}}\pi q + c_{\mathrm{FP}}(1 - \pi)r + \lambda \mathbb{E}D.$$

For a zero-probability class its associated product is zero.

Proof. Expand each indicator expectation as its joint probability and use the definition of conditional probability. Linearity requires no independence. □

This is an accounting identity, not a claim of novelty or a safety bound. A bound $q \leq \alpha$ controls misses among unsafe actions. A different bound $\Pr(Y = 1 \mid A = 1) \leq \alpha$ instead gives

$\Pr(Y = 1, A = 1) \leq \alpha \Pr(A = 1)$, not $\pi\alpha$. Likewise r is a veto rate among safe actions, not the proportion of vetoed actions that are safe. We report all denominators separately.

For bundle-level calibration, u denotes the maximum member score and Y the independently observed bundle outcome. Per-knob labels alone do not establish a bundle-level guarantee.

Selecting a smaller threshold by a cost rule cannot enlarge the acceptance event. Selecting a larger threshold can invalidate the bound; therefore the implementation uses the minimum of the rank threshold and optional cost-selected cap. No unsafe calibration examples implies veto-all. Calibration-window diagnostics are not test-set estimates.

Drift and selective observation. Sliding windows and a drift detector (Bifet and Gavaldà, 2007) do not guarantee exchangeability or a deterministic recovery time. Under an explicitly justified total-variation bound on the *joint calibration–test law*, the probability of the same event changes by at most that bound; a marginal score-distance statement alone does not establish it. We neither estimate that distance nor claim recovery. Online labels selected by acceptance introduce additional selection bias. The runtime holds its threshold at zero on detected change until an explicit refresh; refresh remains an exploratory operation, not proof of safety.

Protocol requirements beyond this study. Workload, hardware and session groups are fixed before graph induction and assigned to disjoint train, retrieval, calibration and test roles; hashes of graphs, sources, prompts and calibrators are frozen before test results are opened, and a validator rejects overlap or unknown lineage. For each bundle we count proposed, rejected, vetoed, accepted, staged, completed and rolled-back outcomes, report gate TP/FP/FN/TN with both #unsafe-accepted/#unsafe and /#accepted, and leave unknown labels unknown; rollback precision needs independently labeled executed actions. A lower threshold accepts a subset of fixed candidates, but the unsafe fraction among survivors need not be monotone. Absolute per-workload metrics precede ratios against a named baseline; a Pareto claim needs several measured operating points per method. Drift studies must keep the threshold trajectory and held-out outcomes at each decision, and faithfulness is tested by deleting cited and matched uncited context under fixed seeds.

Table 5: Follow-up selector replay: mean of ten test-run P95 values. Misses are observed lateness above 200 μs. Shared rows across selectors are intentional: selectors choosing the same configuration use the same held-out observations.

Period	Selector	P95 (μs)	Misses
5	control	179.7	13/400
5	empirical min	179.7	13/400
5	graph min	179.7	13/400
5	seeded random	811.6	248/400
8	control	155.7	13/400
8	empirical min	155.7	13/400
8	graph min	155.7	13/400
8	seeded random	642.2	225/400

B Additional Measurements

B.1 Selector Replay on the Follow-up

All controlled runs used one Intel Core i5-3570 host (four logical CPUs, Linux 6.17.0-23-generic). Each fresh child set its timer slack with `PR_SET_TIMERSLACK` and its affinity with `sched_setaffinity`, checked both by read-back, and recorded 40 wake-ups against absolute monotonic-clock targets after five warm-ups.

The empirical selector minimizes mean training-run P95 over the four configurations. A saturated two-factor representation encodes the same four means as a baseline, two main effects and their interaction; minimizing it adds no information to table lookup. Both selectors have the same 40 training runs and zero online search trials. An unchanged-control selector and a seeded uniform-random selector provide references. Choices are frozen before calibration and test. We evaluate these fixed selectors by paired *offline replay* on the same measured test grid, not by separate closed-loop executions. A block is the uncertainty unit, and paired reductions are calculated within each period against its explicit control.

Both informed selectors chose 50 μs slack and all CPUs, identical to control; their paired improvement is exactly zero in this replay. Control P95 is 179.7 ± 102.0 and 155.7 ± 64.4 μs at 5 and 8 ms (95% Student-t half-width across ten blocks). The training interaction contrast is -18.6 μs; the pre-specified normalized-contrast bootstrap emitted no edge because its interval included zero. We do not turn a nonsignificant contrast into a dependency rule or a semantic-reasoning gain. Acquisition took 6.54 s for training, 8.29 s for retrieval and 10.15 s for calibration; graph construction and selection took 0.10 s, included in calibration-phase wall time.

Table 6: Matched-budget replay: % of 1,000 seeds choosing control / expected paired P95 change at 5 ms (%); last column: mean training evaluations of 1 ms settings at $B = 40$.

Method	$B = 4$	$B = 8$	$B = 16$	$B = 40$	1 ms evals
Random search	62 / −55	74 / −38	85 / −22	98 / −3	19.8
ϵ-greedy (RL)	65 / −51	86 / −20	95 / −8	99 / −1	3.8
UCB1 (RL)	65 / −51	77 / −33	91 / −12	99 / −1	12.0
GP-EI (BO)	65 / −51	83 / −25	95 / −8	99 / −1	2.5

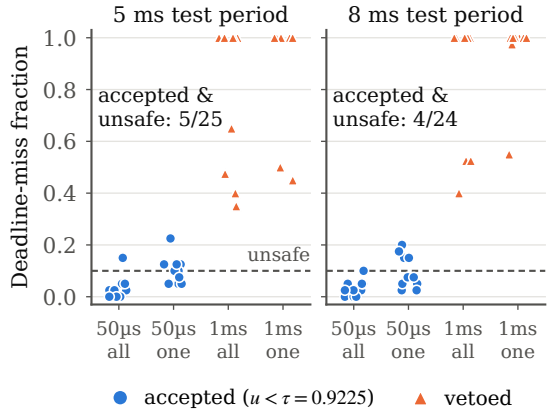


Figure 3: Frozen-gate replay on the 80 held-out test runs; each point is one run, grouped by timer slack (50 μs or 1 ms) and CPU affinity (all CPUs or one). Positive means unsafe (more than 10% of deadlines missed, dashed line) and a gate-positive decision is veto. Counts at every tested α (all give $\tau = .9225$): 5 ms, 20 accepted / 20 vetoed, TP 20, FP 0, FN 5, TN 15; 8 ms, 20 / 20, TP 20, FP 0, FN 4, TN 16. Blue points above the line are the accepted unsafe runs. Generated from raw logs by `plot_gate_replay.py`.

B.2 Matched-Budget Optimizers

Random search, ϵ-greedy Q-learning ($\epsilon = 0.1$), UCB1 and GP-EI BO each evaluate $B \in \{4, 8, 16, 40\}$ recorded training runs, starting with one per configuration, and recommend their lowest observed or posterior mean, scored by the same replay over 1,000 seeds (Table 6). Control is best at both test periods, so this comparison can reveal only losses. With small budgets, noisy single-CPU runs mislead the methods; BO spends fewest evaluations on the 1 ms settings, whose training runs miss over 90% of deadlines.

B.3 Gate Replay

B.4 Staged Actuation

Across 80 fresh executions, 66 stop on a stage breach and 14 complete all stages; all 80 restore and read back both original values. At 5 and 8 ms, observed deadline misses are respectively 186/816 and 190/880; summed exposure durations are 4.106 and 7.066 s. Early stops reduce the number of observed events, so these fractions cannot be compared directly with fixed-duration, no-stop runs to claim an anomaly reduction. The stop criterion itself is not independent ground truth for rollback precision.

Table 7: Measured local baseline. Mean of run medians, and mean of run P95 values with 95% Student-t CI half-width across 10 runs. Last column is the percentage of errors or operations exceeding a predeclared 50 ms deadline. No tuning gain is implied.

Workload	Median ms	P95 ms	Events %
file_fsync	1.903	2.035 ± 0.043	0.00
sqlite_commit	1.870	2.187 ± 0.273	0.00
zlib_compress	1.474	2.038 ± 0.356	0.00

B.5 Local Baseline Measurements

We executed three bounded microbenchmarks on the available host: an Intel Core i5-3570 with four logical CPUs, approximately 31.2 GiB RAM, and Linux 6.17.0-23-generic. These runs characterize the measurement pipeline: no kernel knob was modified and no LLM or controller was compared.

Each workload had 10 separate invocations of 100 measured operations, preceded by 10 warm-up operations. Workload–run order was shuffled with seed 4088. SQLite inserts a 4 KiB value and commits each transaction in WAL mode with synchronous=FULL. The file test overwrites a 64 KiB region and calls fsync. The compression test compresses a fixed seeded 64 KiB payload at zlib level 6 and verifies decompression. Latency is wall-clock operation duration from a monotonic nanosecond clock; the compression measurement includes verification. The raw log retains every operation, run ID, deadline and error flag, with a SHA-256 hash and software/environment manifest.

The anomaly denominator is all measured operations, including failures; an operation counts once if either it fails or exceeds the deadline. This label is independent of veto and rollback. Zero events in these short runs is not evidence of zero population risk. We do not pool heterogeneous workload latencies or infer kernel tuning effects from these baseline timings.

Table 8: Held-out kernel intervention measurements. P95 is the mean of ten run-level P95 wake-up latenesses. Misses count lateness exceeding the predeclared 200 μs deadline. CPU affinity is all four CPUs or one CPU.

Period	Slack/CPUs	P95 (μs)	Misses
2	50us/all	89.5	2/800
2	50us/one	109.4	18/800
2	1ms/all	1016.8	800/800
2	1ms/one	968.9	771/800
5	50us/all	94.2	2/800
5	50us/one	133.8	22/800
5	1ms/all	1022.6	663/800
5	1ms/one	1073.7	680/800

B.6 Temporal-Holdout Kernel Interventions

We additionally executed real Linux timer-slack and CPU-affinity interventions in fresh owned child processes, without modifying host-wide sysctls. The two factors were timer slack (50 μs or 1 ms, set by PR_SET_TIMERSLACK) and affinity (all four available CPUs or CPU 0). Every child checked kernel readback before recording events. This isolated measurement driver is separate from the dry-run GroundKern actuator.

For each periodic wake-up interval (2 or 5 ms), five training blocks and ten subsequent test blocks evaluated all four configurations. Within each block, the eight interval/configuration cases were randomly ordered using seed 4088. Each fresh process performed ten warm-up and 80 measured wake-ups against absolute monotonic-clock targets. The plan was written before execution; the configuration minimizing mean training-run P95 lateness was frozen before test execution. This is temporal holdout within one host and workload family, not the independent workload/hardware split required for claims of generalization. The complete 120-process experiment retained 9,600 event timestamps, readbacks, frozen selections and hashes. No external competing load was introduced.

The training selector chose the explicit control (50 μs slack, all CPUs) at both intervals: there is no selected-policy improvement over that control. Its test P95 was 89.5 ± 9.2 and 94.2 ± 6.9 μs at 2 and 5 ms, respectively (95% Student-t half-width across ten runs). Raising slack to 1 ms with all CPUs increased P95 by 927.3 ± 16.4 and 928.4 ± 10.0 μs in paired blocks. Affinity-only differences were 19.9 ± 36.7 and 39.6 ± 62.6 μs; these intervals include zero. The slack effect is consistent with timer coalescing; no power measurement, graph discovery, LLM advantage or staged rollback is inferred. A miss is defined by observed wake-up time, independently of any safety gate.

C Prompt and Output Schema

Every model receives the following system prompt:

> Select one allowed config to minimize periodic wake-up P95 latency. Return JSON with config (exact allowed name), cited_ids (evidence IDs), and explanation. Evidence contains measured training summaries and retrieval runs, not held-out test outcomes. Do not invent evidence.

The user message is a JSON object with the four allowed configuration names and the evidence list for the input variant. Items have the forms

```
{"id": "training-table", "kind":
"table", "values": {"s50000_aall":
132.9413, ...}}
{"id": "interaction", "kind": "graph",
"contrast_us": -18.62, "edges": [],
"scope": "same measured table; graph is
a reparameterization, not extra
evidence"}
{"id": "retrieval-3-0-s50000_aall",
"kind": "retrieval", "config":
"s50000_aall", "period_ns": 3000000,
"p95_us": 126.36}
```

The output schema requires `config` from the allowed list, `cited_ids` with at most six entries restricted to the supplied IDs (none for the no-evidence variant), and an `explanation` of at most 400 characters. Local models enforce it as a llama.cpp JSON grammar with greedy decoding; hosted models receive it as a strict JSON schema through OpenRouter, restricted to providers that honor it. Every answer is re-validated, and an invented or deleted citation makes it invalid; the explanation length is not re-checked, and 14 of 20 DeepSeek-V3.2 answers exceed 400 characters. Claim and citation counts in §5 are recomputed from these records by `claim_taxonomy.py` and `citation_analysis.py`. A single script, `reproduce_paper.py`, reruns every analysis on a copy of the raw logs and checks that each table and quoted number in the paper is reproduced exactly; the supplementary `ARTIFACTS.md` maps each result to its data and script and includes a data statement. The supplementary records contain each prompt, its SHA-256 hash, the raw response and, for hosted models, the routed provider.

Claim-label checks and intervals. The 97 claim labels were drafted with an AI assistant. A rule-based check (`claim_label_autocheck.py`) re-derives the expected label of 56 claims directly

from the evidence (values attributed to the training table, lists of runs, superlatives, ranges, thresholds, units and no-evidence claims) and agrees with every drafted label; flipping each label between *correct* and *wrong* is detected in 76 of 97 cases. The remaining 41 claims require judgment. One author then reviewed every label against the evidence, reading these 41 claims in full, and changed none (agreement with the draft 100%); because the reviewer checked an existing draft, this is a confirmation rather than an independent annotation. For agreement, two further annotators who are not authors each labeled all 97 claims blind and independently, from the claims, their source sentences and written guidelines, without the drafted labels, checks or each other's labels. The first agrees with the verified labels on 94.8% of claims over the seven labels (Cohen's $\kappa = 0.918$) and 95.9% for misstated versus not ($\kappa = 0.914$); all five disagreements are claims this annotator judged correct (three "significantly lower" statements, one partly wrong claim, one range of cited runs). The second agrees on 90.7% ($\kappa = 0.856$) and 90.7% ($\kappa = 0.808$); five of its nine disagreements read "lowest across all retrieval runs (runs 0–5)" as restricted to runs 0–5, and two arise because the blind sheet showed a sentence from another seed for the claim (since fixed in `claim_review_agreement.py`). Over the verified labels and both annotators, Fleiss' κ is 0.847 (seven labels) and 0.814 (misstated versus not), and the majority label equals the verified label for all 97 claims. The reported counts use the verified labels; with the annotators' labels, 37 and 38 rather than 41 of 97 claims are misstated. With 95% Wilson intervals, the misstated proportions are 57% (39–73%) for local models, 36% (26–48%) for hosted models and 42% (33–52%) overall; 97% (94–98%) of cited retrieval runs belong to the chosen configuration and 80% (75–85%) come from the first half of the evidence list. These intervals treat claims and citations as independent, which overstates their precision because hosted explanations repeat claims across seeds. Two dependence-aware checks give similar values: counting each distinct claim of a model once, 25 of 67 (37%) are misstated, and a bootstrap that resamples whole models gives 42% (34–52%) overall.

Entailment-based attribution judges. To test whether citation-support judges catch these errors, `entailment_judge.py` gives each of the 93 evidence-based claims to eight judges with pinned revisions: three NLI cross-encoders (DeBERTa-v3-large trained on five NLI sets, DeBERTa-v3-large trained on SNLI/MNLI, RoBERTa-large-MNLI; *supported* = entailment), Flan-T5-large asked whether the claim is fully supported (*supported* = yes), and four judges trained to fact-check claims against grounding documents, AlignScore-large (Zha et al., 2023) and MiniCheck with RoBERTa-large, DeBERTa-v3-large and Flan-T5-large backbones (Tang et al., 2024) (*supported* = score above 0.5). As in citation-support evaluation, the premise is the verbalized evidence the model cited; the hypothesis is either the explanation sentence that states the claim or the extracted claim. Table 9 compares the judges with the author-verified labels. No judge separates misstated from correct claims well (balanced accuracy 0.47–0.68), and at least one judge accepts 28 of the 41 misstated claims. Lenient judges accept up to half of the wrong values, ranges and units, the near-misses that quote correct numbers; strict judges reject nearly all overstatements but also most correct claims. The fact-checkers are the strictest: MiniCheck accepts at most 3 of 51 correct claims. Correct claims about uncited evidence cannot be entailed by the cited items alone, but giving the two long-context MiniCheck judges all 42 evidence items as premise (claim*) still leaves them near chance (0.49 and 0.52), because claims such as ranges and orderings require aggregating many items. TRUE (T5-11B) did not fit the 15 GB of memory of our CPU container and was not tested.

Compute budget. Local models ran as 4-bit GGUF files (Q4_K_M; Phi-4 Q4_K) with llama-cpp-python 0.3.35 (`https://github.com/abetlen/llama-cpp-python`) on a four-core CPU container without a GPU, with an 8,192-token context and at most 384 output tokens. Their 100 calls took 11,580 s of wall time (about 3.2 CPU-hours): Qwen2.5-7B 1,603 s, Llama-3.1-8B 2,325 s, Qwen2.5-14B 3,136 s and Phi-4 4,516 s. The 150 hosted calls through OpenRouter, with at most 4,096 output tokens, took 1,224 s in total and cost about 0.5 USD; the 150 robustness calls (plus two retries) cost 0.83 USD over about 2.5 h of wall time, the 90 mitigation calls (plus five retries) 0.70 USD, the 40 reasoning-toggle calls (20 in a discarded first attempt) 0.18 USD, the 90 evidence-instance calls 0.45 USD, and the 65 database-task calls 0.28 USD. The eight attribution judges ran

Table 9: Off-the-shelf attribution judges on the 93 evidence-based claims: claims accepted as supported among wrong values (wrong, wrong unit, wrong attribution; 22), overstatements (19) and correct claims (51), with balanced accuracy (BA) against the verified labels. Hyp.: hypothesis is the source sentence or the extracted claim.

Judge	Hyp.	Wrong	Over.	Correct	BA
DeBERTa-v3 (5 NLI sets)	sent.	7/22	3/19	16/51	0.54
DeBERTa-v3 (5 NLI sets)	claim	11/22	1/19	29/51	0.64
DeBERTa-v3 (SNLI/MNLI)	sent.	3/22	0/19	11/51	0.57
DeBERTa-v3 (SNLI/MNLI)	claim	3/22	0/19	19/51	0.65
RoBERTa-large-MNLI	sent.	1/22	0/19	4/51	0.53
RoBERTa-large-MNLI	claim	3/22	0/19	13/51	0.59
Flan-T5-large	sent.	6/22	8/19	18/51	0.51
Flan-T5-large	claim	9/22	4/19	34/51	0.68
AlignScore-large	sent.	5/22	0/19	12/51	0.56
AlignScore-large	claim	6/22	0/19	12/51	0.54
MiniCheck-RoBERTa	sent.	2/22	0/19	1/51	0.48
MiniCheck-RoBERTa	claim	3/22	0/19	3/51	0.49
MiniCheck-DeBERTa	sent.	1/22	0/19	0/51	0.49
MiniCheck-DeBERTa	claim	3/22	0/19	1/51	0.47
MiniCheck-Flan-T5	sent.	1/22	0/19	0/51	0.49
MiniCheck-Flan-T5	claim	2/22	0/19	0/51	0.48
MiniCheck-DeBERTa	claim*	5/22	0/19	5/51	0.49
MiniCheck-Flan-T5	claim*	5/22	0/19	8/51	0.52

on the same CPU container. Kernel measurements ran on the Intel Core i5-3570 host described in Appendix B; acquiring the training, retrieval and calibration roles took 6.54, 8.29 and 10.15 s. Optimizer replays over 1,000 seeds take seconds on one CPU.

D Results by Model Scale and Family

`scale_family_analysis.py` re-analyses the existing records by scale and family without new model calls (Table 10). It uses only full-context answers, since local models were also audited on ablated prompts; claims of local models are counted once per distinct explanation because their greedy outputs repeat across seeds, whereas hosted explanations vary and repeat claims across seeds. Among the eight open-weight models, the misstated share falls with log total parameters (Spearman $\rho = -0.78$, $p = 0.02$), and local models misstate more often than hosted open-weight models (7/8 versus 21/52; Fisher's exact test $p = 0.02$). With eight local claims and repeated hosted claims, this is a trend, not an estimate. The error types change with scale: wrong units and source attributions occur only in 7–15B models, whereas overstatements and wrong values occur at every scale and in every family. Citation behavior does not track scale: every open-weight model cites only runs of its chosen configuration; Gemini-Pro is the only model that cites runs of other configurations (8 of 21), and front-loading is strongest in local models (100 of 110 cited runs in the first half) and weakest in Llama-70B (14 of 28). Within the Qwen and Llama families, the larger model misstates a smaller share (Qwen 2/2 and 2/2 at 7B and 14B versus 3/6 at 235B; Llama 2/2 versus 2/7).

E Robustness to Prompt, Evidence Order, Citation Cap and Decoding

`prompt_robustness.py` reruns the full-context prompt of the six hosted models, five seeds each, with the same frozen evidence, strict JSON schema and OpenRouter settings as the main audit, under five conditions: a paraphrased system prompt; retrieval runs in reverse order; retrieval runs in a seed-specific random order; no citation cap (any number of supplied IDs); and sampling at temperature 0.7. All 150 answers are valid after one retry of two answers that a provider truncated. Table 11 reports the results. Every answer in every condition chooses the selector's configuration, and every cited run belongs to the chosen configuration (in the original runs only Gemini-Pro cites others). Reversing the runs moves citations with list position for four models (Llama-70B, Qwen3-235B and both Gemini models), whereas Nemotron keeps citing blocks 0–4 and DeepSeek-V3.2 falls in between; without the cap, four models cite all ten runs of the chosen configuration. The explanations of these new answers have no verified claim labels, so we score them with two automatic proxies, applied identically to the original runs: a universal quantifier attached to a comparison ("consistently the lowest", "lower in all runs") or "significantly lower", which the evidence does not support; and decimal numbers that match no evidence value or derived minimum, maximum, mean or difference. The quantifier overstatement recurs in every condition (9 to 21 of 30 explanations, 21 in the original runs), while unsupported numbers are rare (at most two per condition), consistent with the verified finding that misstated claims mostly quote correct numbers.

F Mitigation Baselines

`mitigation_baseline.py` reruns the full-context prompt of the six hosted models, five seeds each, with the frozen evidence, schema, citation cap and OpenRouter settings of the main audit, under three mitigations: an instruction added to the system prompt to check every number, range, extreme and

Table 10: Full-context results by model: answers choosing the selector's configuration (Sel.), cited retrieval runs of the chosen configuration and from the first half of the evidence list, misstated claims, and misstated claims by type (overstated / wrong value / wrong unit or source). Gemini parameter counts are undisclosed.

Model	Params	Sel.	Cited chosen	First half	Misst.	Over./Val./Unit
Qwen-7B	7.6B	5/5	25/25	25/25	2/2	1/0/1
Llama-8B	8B	0/5	30/30	25/30	2/2	0/2/0
Qwen-14B	14.7B	5/5	25/25	25/25	2/2	1/0/1
Phi-4	14.7B	5/5	30/30	25/30	1/2	1/0/0
Local open, 7–15B		*15/20*	*110/110*	*100/110*	*7/8*	*3/2/2*
Llama-70B	70.6B	5/5	28/28	14/28	2/7	0/2/0
Qwen3-235B	235B	5/5	30/30	25/30	3/6	3/0/0
DeepSeek-V3.2	671B	5/5	25/25	25/25	6/16	2/4/0
Nemotron-550B	550B	5/5	30/30	25/30	10/23	5/5/0
Hosted open, 70–671B		*20/20*	*113/113*	*89/113*	*21/52*	*10/11/0*
Gemini-Flash	–	5/5	16/16	10/16	2/12	2/0/0
Gemini-Pro	–	5/5	13/21	10/21	2/5	2/0/0
Proprietary		*10/10*	*29/37*	*20/37*	*4/17*	*4/0/0*

Table 11: Hosted models under prompt, order, cap and decoding variations (30 answers per condition): answers choosing the selector's configuration (Sel.), mean citations per answer, cited retrieval runs of the chosen configuration, in the first half of the list as presented, and from blocks 0–4, and explanations with a quantifier overstatement (automatic proxy).

Condition	Sel.	Cites	Chosen	First half	Blocks 0–4	Overst.
Original (greedy)	30/30	5.5	142/150	109/150	109/150	21/30
Paraphrased prompt	30/30	4.8	131/131	94/131	94/131	15/30
Reversed runs	30/30	5.1	138/138	88/138	50/138	13/30
Shuffled runs	30/30	5.1	138/138	76/138	67/138	9/30
No citation cap	30/30	9.0	256/256	128/256	128/256	21/30
Sampling (T=0.7)	30/30	5.1	138/138	94/138	94/138	13/30

comparison against the evidence, to avoid quantifiers the evidence does not support and to cite evidence for other configurations as well (*verify*); one deterministic summary item per configuration added to the evidence, with the minimum, maximum and mean of its retrieval runs and the number of blocks in which it is lowest, a tool-style aid that removes hand aggregation (*stats*); and both. All 90 answers are valid after one retry of five. These explanations have no verified claim labels, so Table 12 scores them with automatic checks applied identically to the 30 original hosted answers: the quantifier-overstatement proxy of Appendix E; an exact range check, which flags a stated range for a configuration that equals neither the minimum and maximum of its runs nor those of the runs the answer cites; and unsupported decimal numbers. On the original answers, the range check flags exactly the seven explanations that contain a verified wrong range and no other, and the proxy fires for exactly the five models with verified overstatements.

Every answer in every condition still chooses the selector's configuration. The verify instruction removes almost all flagged quantifiers (21 to 1 of 30 explanations) and most range errors (7 to 2), but only the two Gemini models begin citing competing runs. Summary statistics remove range errors and lead 19 of 30 answers to cite a competing configuration's summary, yet four answers (three from Qwen3-235B, one from DeepSeek-V3.2) still call the control "consistently" lowest or the others "significantly" higher, although the supplied summary says it is lowest in 8 of 10 blocks; the one flagged verify answer restricts "every run" to runs 0–5. With both, no explanation is flagged. The checks see only two error types: reading every explanation for universal comparisons found residual errors that they miss, among them a verify answer stating that only runs 0–5 contain all four configurations (all ten blocks do) and a stats answer stating that all other configurations are typically above 748 μs (`s50000_aone` averages 227 μs); the two remaining range flags under verify are both genuine. These mitigations therefore reduce the checkable errors substantially but do not certify an explanation; verified labels for the 90 explanations

(exported for review with the records) are needed to measure their full effect.

Table 12: Hosted models with mitigations (30 answers per condition): answers choosing the selector's configuration (Sel.), cited runs or summaries of a competing configuration among all cited runs and summaries, and explanations flagged by the quantifier-overstatement proxy, the exact range check and the unsupported-number check. Automatic checks, not verified labels.

Condition	Sel.	Contrary cites	Overst.	Range err.	Unsupp. num.
Original prompt	30/30	8/150	21/30	7/30	0/30
Verify instruction	30/30	19/146	1/30	2/30	0/30
Summary statistics	30/30	42/102	4/30	0/30	0/30
Both	30/30	63/124	0/30	0/30	0/30

G Reasoning Models

In the main audit, the two Gemini models reasoned internally by default (about 490 and 1,100 hidden reasoning tokens per answer for Flash and Pro); the other hosted models produced none. `reasoning_models.py` toggles reasoning within a model on the full-context prompt, five seeds each, and compares with the same model's original answers: reasoning enabled for DeepSeek-V3.2 and Nemotron-3-Ultra-550B, the thinking variant of Qwen3-235B-A22B (Thinking-2507), and minimal reasoning effort for Gemini 3.8 Flash, whose endpoint refuses to disable reasoning. A first attempt showed that some providers silently drop reasoning when a strict JSON schema is requested, so an answer counts only if the requested reasoning was spent; DeepSeek-V3.2 is pinned to a provider that applies both, while Nemotron and the Qwen3 thinking variant, whose endpoints do not combine them, receive the schema in the prompt and are validated with the same rules. All 20 answers are valid and every reasoning arm spent 585 to 5,306 reasoning tokens.

Table 13 reports the automatic checks of Appendix F. Reasoning changes no decision and does not make any model cite a competing configuration. It does not remove the quantifier overstatement: DeepSeek-V3.2 and the Qwen3 thinking variant still call the control "consistently" lowest in four and five of five explanations. Nemotron with reasoning makes fewer flagged errors (two overstatements and one wrong range, down from five each), but its explanations are also about half as long (190 versus 400 characters), so fewer claims are made; Gemini Flash with minimal reasoning flags one explanation instead of two. These checks see only two error types and the answers have no verified labels, so the comparison shows that internal reasoning is not sufficient to remove the errors, not how many remain.

Table 13: Within-model reasoning toggles (five answers each, original → toggled): mean reasoning tokens per answer, answers choosing the selector's configuration, cited runs of a competing configuration, and explanations flagged by the quantifier-overstatement proxy and the exact range check. Automatic checks, not verified labels.

Model (toggle)	Reas. tok.	Sel.	Contrary	Overst.	Range err.
DeepSeek-V3.2 (on)	0→3493	5→5/5	0→0	4→4	0→0
Nemotron-550B (on)	0→1706	5→5/5	0→0	5→2	5→1
Qwen3-235B (thinking)	0→1123	5→5/5	0→0	5→5	0→0
Gemini-Flash (minimal)	493→0	5→5/5	0→0	2→1	0→0

H Precision of the Local Models

The local models of the main audit ran as 4-bit GGUF files. `precision_check.py` compares them with higher-precision reruns through the same `model_audit.py` pipeline, with all input variants, cited and matched uncited deletion, five seeds, temperature 0 and the same 384-token limit: Llama-3.1-8B and Phi-4 with bf16 weights through OpenRouter, pinned to providers that declare bf16 (CoreWeave and DeepInfra), and Qwen2.5-7B as the official 8-bit (Q8_0) GGUF in llama.cpp, the highest precision that fits our 15 GB CPU machine. Qwen2.5-14B has no full-precision endpoint and was not rerun. The bf16 runs use a different serving stack and strict JSON schemas rather than a llama.cpp grammar, so any difference may come from precision or from serving.

Table 14 summarizes the comparison. In every evidence and no-evidence variant, each model makes the same choice as at 4 bits on every valid seed, including Llama-3.1-8B's wrong choice of `s50000_aone` and the no-evidence choices; one bf16 Phi-4 answer without evidence is invalid. Explanation errors recur: Qwen2.5-7B again attributes the retrieval value 126.36 to the training table and again reads meanings into the configuration names without evidence; Phi-4 repeats its "significantly lower" claims almost verbatim; and Llama-3.1-8B, whose 4-bit explanations misstated values, now calls a list of single runs an average and claims the lowest average "across all retrieval runs". The one difference concerns reliance. With bf16 weights, Phi-4 cites the same six runs, but deleting them no longer changes its choice in any

seed, whereas the 4-bit model switched in all five. Llama-3.1-8B changes its choice under both deletions at either precision (to a different wrong setting under cited deletion). The paper's only case of citation-specific reliance therefore does not survive a change of precision and serving stack.

Table 14: Local models at 4 bits versus higher precision: input variants with the same choice on every common valid seed, seeds whose choice changes under cited and under matched uncited deletion (4-bit → higher precision; n/e: non-estimable), and full-context explanations flagged by the quantifier-overstatement proxy.

Model	Precision	Same choice	Del. cited	Del. unc.	Overst.
Qwen2.5-7B	8-bit (Q8_0)	4/4	n/e	n/e	5→5/5
Llama-3.1-8B	bf16	5/6	5→5/5	5→5/5	0→5/5
Phi-4	bf16	5/6	5→0/5	0→0/5	5→5/5

I Redesigned Reliance Test

The main audit's matched deletion has two weaknesses: the training table is a single item, so models that cite it have no same-kind control, and the uncited controls are runs of other configurations, so the two deletions differ in what the removed evidence says. `reliance_redesign.py` reruns the test on the same frozen evidence, five seeds per model, for the six hosted models and for Llama-3.1-8B and Phi-4 with bf16 weights (Appendix H). The training table is split into four per-configuration items, and two deletion tests follow each full-context answer: (A) the main audit's kind-matched test, now estimable when a table item is cited; and (B) a configuration-matched test that deletes k cited retrieval runs of the chosen configuration versus k uncited runs of the same configuration, with k the smaller count (4–5 here). Controls are drawn with a seed-specific RNG.

Splitting the table leaves every hosted model's choice unchanged but turns bf16 Llama-3.1-8B from `s50000_aone` to the control in all five seeds, so its earlier wrong choice depends on how the table is presented. Table 15 gives the outcomes. Test A is estimable for 32 seeds and test B for 35; Gemini Flash cites all four table items, which leaves no table control. Under test B no hosted model and not Phi-4 ever changes its choice, while bf16 Llama-3.1-8B switches to `s50000_aone` in all five seeds when its cited runs (blocks 0–4 of the control) are deleted and keeps the control when five uncited control runs are deleted. Test A agrees for Llama-3.1-8B and adds one DeepSeek-V3.2 seed, but its cited deletion removes the chosen configuration's own table entry while the control removes another configuration's, so test A still confounds citation with content. Even test B leaves different blocks in the prompt (in blocks 5–9 the competitor wins twice), so a change under cited deletion shows sensitivity to the cited items, not that the model's computation used them. Together with Appendix H, these results show that citation-specific reliance appears in at most one model per design and that which model shows it depends on weight precision and on how the evidence is formatted.

Table 15: Redesigned deletion test (five seeds per model, split training table): full-context answers citing a table item, and estimable seeds whose choice changes under cited deletion only / both / uncited deletion only / neither, for the kind-matched test A and the configuration-matched test B.

Model	Cites table	A: kind-matched	B: config-matched
Llama-3.3-70B	1/5	0/0/0/5	0/0/0/4
Qwen3-235B	5/5	0/0/0/5	0/0/0/5
DeepSeek-V3.2	5/5	1/0/0/4	0/0/0/5
Nemotron-550B	5/5	0/0/0/5	0/0/0/5
Gemini-Flash	5/5	n/e	0/0/0/2
Gemini-Pro	5/5	0/0/0/2	0/0/0/5
Llama-8B (bf16)	5/5	5/0/0/0	5/0/0/0
Phi-4 (bf16)	5/5	0/0/0/5	0/0/0/4

J Further Evidence Instances

The main audit gives every model one evidence list. To test whether the findings depend on it, `evidence_instances.py` reruns the six hosted models (same system prompt, schema, six-citation cap, temperature 0, five seeds; 90 calls, 0.45 USD) on three further instances built only from training-side data, so no test outcome enters a prompt. *I1* keeps the training table but replaces the retrieval runs with the 40 calibration runs at 4 ms; there the table's choice is lowest in only 5 of 10 blocks, and `s50000_aone` has the lowest single run but a run mean twice as high (263.0 versus 121.5 μs). *I2* is the main evidence with the four labels permuted, so the best configuration is named `s1000000_aone`, the name that suggests the worst setting. *I3* comes from the separate measurement session of Appendix B: a table from its five training blocks at 2 ms and its 20 training runs at 5 ms as retrieval items. The graph item is omitted. Answers are scored with the automatic checks of Appendix F against each instance's own evidence. The quantifier proxy counts "consistently"/"all" lowest only if the chosen configuration is not lowest in every

block and ignores "lowest average"; "significantly" always counts, since the evidence contains no test. These explanations have no verified claim labels.

Table 16 gives the results. Every answer is schema-valid. With I2 all 30 answers choose the relabeled best configuration, so the choice follows the evidence, not the names. With I3 all 30 choose the table's choice. With I1, where the runs disagree with the table more often, Llama-3.3-70B chooses `s50000_aone` in all five seeds and Qwen3 in one. Llama-3.3-70B's explanations claim that configuration has the lowest average, and Qwen3's that it is consistently lowest; neither the table nor the runs support either claim. So the evidence suffices for a table-lookup choice only when the runs do not compete with the table. One-sided citation persists in every instance: only Gemini Flash cites competing runs (6 of 155 in I1). Front-loading cannot be measured in I3, where five runs per configuration fit under the cap. The quantifier proxy fires in 13–18 of 30 explanations per new instance, against 21 in the main audit. In I2 "consistently the lowest" recurs for the relabeled configuration, while the range errors of the main audit disappear: Nemotron now gives the competing range correctly.

The range check flags six I1 and four I3 explanations. On reading, eight contain a wrong range: `s50000_aone` given as 76.186–148.876 or 76.186–697.378 (its maximum is 705.847), the chosen configuration given as 85.9–129.5 (it reaches 312.316), and two run values given as the range of `s50000_aone` in I3. The other two are acceptable approximations ("∼900–1500" and "∼85–102 across most runs"). The proxy also misses phrasings such as "across all five retrieval runs". In I3, Llama-3.3-70B averages the five chosen-configuration runs and reports 93.3188, 93.3392 or 93.39 μs; the mean is 93.339, so the computed values in its explanations are close but wrong. The reading was done with an AI assistant and has not been independently verified; `explanations-for-review.csv` lists every explanation with its flags. All three instances come from the same host and workload family as the main audit, so they test robustness to the evidence values, their order and the labels, not to other domains.

Table 16: Hosted models on further evidence instances (six models, five seeds): blocks in which the table's choice has the lowest run, answers choosing it, cited retrieval runs of a competing configuration and from the first half of the list, and explanations flagged by the quantifier proxy and the range check (automatic checks, not verified labels). The main row re-scores the main audit's answers with the same checks.

Evidence	Sel. wins	Sel. choice	Contrary	First half	Overst.	Range err.
Main (3 ms runs)	8/10	30/30	8/150	109/150	21/30	7/30
I1: 4 ms runs	5/10	24/30	6/155	113/155	13/30	6/30
I2: labels permuted	8/10	30/30	0/152	104/152	18/30	0/30
I3: second session	4/5	30/30	0/148	59/148	13/30	4/30

K A Second Domain: Database Settings

Much LLM configuration work tunes databases (Trummer, 2022; Lao et al., 2024; Giannakouris and Trummer, 2025). `domain_sqlite.py` therefore repeats the audit on a database task measured for this purpose. Four configurations cross SQLite's `synchronous` setting (NORMAL or OFF) with its journal mode (DELETE or WAL). The objective is the P95 commit latency of write transactions of ten 100-byte rows. Each run is a fresh process with a fresh database file and records 10 warm-up and 200 measured transactions; the readback confirms every setting. Training, retrieval and held-out test roles of ten blocks each are separated in time, with configurations randomized within blocks under a plan written before measurement (SQLite 3.45.1, the four-core container of Appendix C). The evidence has the same format as the kernel task: a training table and 40 retrieval runs in block order. The task differs in three ways: the labels are descriptive, so priors about SQLite apply; the evidence sources disagree; and the measurements were committed before any model call. The table favors `sync_off_wal` (71.7 versus 99.5 μs for `sync_normal_wal`). In the retrieval runs, `sync_normal_wal` has the lower mean (61.7 versus 64.3 μs) and wins 8 of 10 blocks. On the test blocks, `sync_off_wal` has the lower mean P95 (47.4 versus 56.0 μs) and wins 7 of 10. Its paired advantage is $20.6 \pm 26.7\%$, so the two WAL settings are not reliably separated.

The six hosted models answered the main audit's prompt and schema, with only the objective changed, under the full evidence and with no evidence (five seeds each; 0.28 USD including retries). Three full-evidence answers stayed invalid after one retry (two truncated, one over the citation cap). Table 17 gives the choices. Without evidence, 25 of 30 answers choose `sync_off_wal`, the test-best setting, which reflects common knowledge about SQLite. With evidence, 21 of 27 choose

sync_normal_wal and so follow the retrieval runs rather than the table. Only Gemini 3.1 Pro (five seeds) and one Gemini Flash seed keep the table's choice. The evidence thus changes the choice away from both the prior and the table, toward a setting that is not better on the test blocks. Citations are again selective. The 21 answers choosing sync_normal_wal cite no run of another configuration, and Gemini 3.1 Pro cites competing runs only from the two blocks in which its choice wins.

We read all 27 explanations against the evidence with an AI assistant; the reading has not been independently verified (explanations-read.csv). Thirteen contain at least one misstatement, all among the 21 answers choosing sync_normal_wal, and none were found in the six answers that keep the table's choice. The dominant error is new. Nine explanations misreport the training table so that it agrees with the choice: three Nemotron answers call sync_normal_wal lowest "across both training and retrieval evidence", two Nemotron answers call 99.496 μs "significantly lower" than 71.652 μs, and four Qwen3 or DeepSeek-V3.2 answers call it the table's lowest or better than sync_off_wal. Other errors repeat the kernel task's patterns. Five explanations claim the choice is "consistently" lowest (it wins 8 of 10 blocks). Four give a wrong range for sync_normal_wal, each omitting its 114.05 μs run in block 3. One states a wrong median and one uses milliseconds. The automatic checks see only part of this: the quantifier proxy flags 9 explanations and the range check 4. The range check partly flags ranges that are correct for the stated blocks and misses one wrong range, and neither check detects the misreported table. In this domain, then, the decision departs from the equally informed selector, and the rationales misreport the evidence that contradicts the decision.

Table 17: SQLite task: choices of the table selector and of the hosted models with full evidence and without evidence (superscripts count valid seeds). N/O: synchronous NORMAL/OFF; D/W: journal DELETE/WAL. The last rows give each other configuration's paired test P95 change versus the selector's choice (95% half-width over ten blocks).

Model	Full evidence	No evidence
Table selector	O/W	–
Llama-3.3-70B	N/W^5	O/W^5
Qwen3-235B	N/W^5	O/W^5
DeepSeek-V3.2	N/W^4	N/D^2 O/W^3
Nemotron-3-Ultra-550B	N/W^5	N/W^3 O/W^2
Gemini-3.8-Flash	N/W^2 O/W^1	O/W^5
Gemini-3.1-Pro	O/W^5	O/W^5
Test P95 vs. selector's choice (%):		
N/D	$+3922.8 \pm 499.8$	
N/W	$+20.6 \pm 26.7$	
O/D	$+200.2 \pm 61.4$	